

This section describes concepts that are relevant across multiple parts of the Teaching Vacancies system. They are not confined to a single building block but affect the design and implementation of many components.

8.1. Authentication and Authorization

- **Description:** Authentication is the process of verifying the identity of a user (Jobseeker, Publisher, Support User). Authorization is the process of determining whether an authenticated user has permission to access a specific resource or perform a particular action. This is critical for protecting user data and ensuring users only access appropriate functionalities.
- **Approach/Technologies:**
- **Core Authentication:** Django's built-in authentication system (`django.contrib.auth`) with the custom `users.User` model (`AUTH_USER_MODEL`).
- **Jobseeker Authentication:** Primarily via GOV.UK One Login (using OpenID Connect). `django-allauth` will be used to manage the OIDC client-side flow and link One Login identities to `users.User` accounts. Fallback mechanisms (e.g., magic links via email) might be considered if One Login is unavailable.
- **Publisher & Support User Authentication:** Via DfE Sign-In (DSI) (using OAuth 2.0 / OpenID Connect). `django-allauth` will manage the OIDC/OAuth client-side flow and link DSI identities to `users.User` accounts.
- **Local Development:** `django-allauth` will also facilitate standard email/password-based login for local development and testing across all user types.
- **Authorization:** Django's built-in permission framework (groups and permissions). Permissions will be assigned to user groups (e.g., "Publishers", "Jobseekers", "Support Level 1") or directly to users based on their `user_type` and potentially other attributes (e.g., organisation membership for publishers). Custom permission classes or decorators might be used in views.
- **Impact:**
- **users app:** Defines the `User` model and `UserManager`.
- **jobseekers, publishers apps:** Views in these apps will use decorators like `@login_required` and permission checks (e.g., `user.has_perm()`, custom permission classes) to protect access.
- **All apps with views:** Need to consider the authentication status and authorization level of the user.

- **django-allauth:** Provides templates, views, and signals for authentication flows.

8.2. Data Encryption

- **Description:** Protecting sensitive Personally Identifiable Information (PII) and other confidential data stored in the database (at rest) and when transmitted over the network (in transit).
- **Approach/Technologies:**
- **At Rest:** Use of field-level encryption for specific model fields containing PII. The `django-cryptography` library was chosen during implementation attempts after issues with other libraries. This library encrypts field values before saving them to the database and decrypts them when accessed. A strong encryption key (e.g., `CRYPTOGRAPHY_KEY` in settings, managed via environment variables) is essential.
- **In Transit:** HTTPS/TLS will be enforced for all communication between clients (browsers, API consumers) and the application servers, and between application components and external services where possible. This is typically configured at the load balancer/web server level.
- **Impact:**
- **users.User model:** Fields like `family_name`, `given_name` (for publishers) were identified for encryption.
- **job_applications models (e.g., JobApplication, Referee):** Numerous fields containing applicant PII will use encrypted fields.
- **jobseekers models (e.g., PersonalDetails):** Fields like `TRN`, names, phone numbers will use encrypted fields.
- **Configuration:** Requires `CRYPTOGRAPHY_KEY` to be securely managed and deployed.
- **Database:** Encrypted data is stored as ciphertext in standard database columns (e.g., `TextField`, `CharField`). Querying encrypted fields directly with database functions (e.g., `WHERE encrypted_field = 'value'`) is generally not possible; filtering must happen after decryption in application code or by using specialized database extensions if available and appropriate.

8.3. Background Task Processing

- **Description:** Offloading long-running or resource-intensive operations from the synchronous request-response cycle to improve web application

responsiveness and handle tasks that don't require immediate user feedback.

- **Approach/Technologies:**
- **Celery:** A distributed task queue system will be used to manage and execute background tasks.
- **Redis:** Will serve as the message broker for Celery (to queue tasks) and potentially as a results backend (to store task status/results if needed).
- **Celery Beat:** Will be used for scheduling periodic tasks (e.g., daily job alerts, data cleanup routines).
- **Impact:**
- **notifications app:** Sending emails (especially bulk job alerts) will be done via Celery tasks.
- **vacancies app:** Tasks like updating search indexes after vacancy changes, or exporting vacancies to DWP Find a Job.
- **organisations app:** Data synchronization with GIAS.
- **core app (or dedicated tasks app):** May house general maintenance tasks (e.g., deleting old data, generating reports).
- **Deployment:** Requires separate Celery worker and Celery Beat processes/containers in the deployment environment.

8.4. Email Handling

- **Description:** Sending various types of emails to users, including transactional emails (account confirmations, password resets), job alerts, application status updates, and potentially newsletters or announcements.
- **Approach/Technologies:**
- **GOV.UK Notify:** The primary service for sending emails, using the `notifications-python-client` library. This ensures compliance with government communication standards and provides robust delivery, tracking, and templating.
- **Django's Email System (potentially as a wrapper):** Django's mail functions might be used to structure email content or manage templating, with the actual sending delegated to a GOV.UK Notify backend.
- **Celery:** Most email sending (especially non-critical or bulk emails like job alerts) will be handled asynchronously by Celery tasks to avoid blocking web requests.

- **Templating:** Django’s template engine or GOV.UK Notify’s own templating features will be used for email content. Email templates will adhere to GOV.UK Design System and accessibility guidelines.
- **Impact:**
- **notifications app:** Centralizes the logic for preparing and dispatching emails via Celery tasks and GOV.UK Notify.
- **users app:** For account-related emails (password reset, email confirmation via `django-allauth` which needs Notify integration).
- **jobseekers (Subscriptions):** For sending job alert emails.
- **job_applications:** For sending application status updates.
- **Configuration:** Requires GOV.UK Notify API keys and template IDs to be configured.

8.5. Search Functionality

- **Description:** Providing users (primarily jobseekers) with effective tools to find relevant vacancies, and potentially for publishers or support users to find organisations or users.
- **Approach/Technologies:**
- **PostgreSQL Full-Text Search:** Utilized via `django.contrib.postgres.search.SearchVectorField` on models like `Vacancy` and `Organisation`. This allows for keyword-based searching on indexed content. `SearchQuery` and `SearchRank` will be used to build search queries and rank results.
- **GeoDjango for Location-based Search:** For searching vacancies or organisations within a certain radius of a location. This uses PostGIS functions via Django’s ORM (e.g., `dwithin` for radius searches). Requires `PointField` on models like `Vacancy` and `Organisation`. (Note: During initial model implementation, `PointField` was temporarily placeholder due to GDAL issues in the sandbox).
- **Filtering:** Django-filter or custom filter logic in views/services will be used for faceted search (e.g., filtering by phase, working pattern, contract type).
- **Service Layer:** Search logic will be encapsulated in service classes or model manager methods to keep views clean and logic reusable.
- **Impact:**
- **vacancies.Vacancy model:** Contains `searchable_content` (`SearchVectorField`) and `geopoint` (`PointField`, currently placeholder).

- **organisations.Organisation model:** Contains `searchable_content` and `geopoint`.
- **vacancies.views / organisations.views:** Handle search forms and orchestrate search execution.
- **Database:** Requires PostgreSQL with PostGIS extension. `SearchVectorField` requires database triggers or application-level logic (signals/tasks) to keep it updated.

8.6. API Design and Provision

- **Description:** Exposing system functionalities and data to external consumers (e.g., Applicant Tracking Systems) or potentially for a decoupled frontend in the future.
- **Approach/Technologies:**
- **Django REST framework (DRF):** The primary framework for building web APIs.
- **Serializers:** DRF serializers (`ModelSerializer`, custom serializers) will be used to convert Django models and other data structures to JSON representations and validate incoming API data.
- **ViewSets & Views:** DRF `ViewSet`s` (especially `ModelViewSet`) or standard `APIView` classes will be used to define API endpoints and their behavior.
- **Authentication:** Token-based authentication (e.g., DRF's `TokenAuthentication` or API Keys) for external systems like ATS providers.
- **Permissions:** DRF permission classes (e.g., `IsAuthenticated`, custom permissions) to control access to API resources.
- **Versioning:** APIs will be versioned (e.g., `/api/v1/...`) to allow for future changes without breaking existing integrations.
- **Documentation:** API documentation will be generated, potentially using tools like `drf-spectacular` or `drf-yasg` to create OpenAPI (Swagger) specifications.
- **Impact:**
- **api app:** Contains API-specific views, serializers, and URL configurations.
- **Models in other apps (vacancies, organisations, etc.):** Serializers in the `api` app will reference these models.
- **Security:** API endpoints need robust authentication and authorization.

8.7. Error Handling and Logging

- **Description:** Gracefully managing errors that occur during application runtime and providing comprehensive logging for debugging, monitoring, and auditing purposes.
- **Approach/Technologies:**
- **Django’s Exception Handling:** Standard Django mechanisms for handling exceptions in views (e.g., `try...except` blocks, custom exception classes, `Http404`).
- **Custom Error Pages:** User-friendly error pages for common HTTP errors (404 Not Found, 500 Internal Server Error, 403 Forbidden) will be implemented, styled according to GOV.UK Design System.
- **Structured Logging:** Use of `python-json-logger` or similar to produce logs in a structured format (JSON), making them easier to parse, search, and analyze by logging platforms.
- **Logging Configuration:** Django’s logging settings will be configured to define log levels, handlers (console, file, external services), and formatters.
- **External Logging/Monitoring Services:** Integration with services like Sentry for error tracking and aggregation, and a centralized logging platform (e.g., ELK stack, CloudWatch Logs, Papertrail) for log storage and analysis.
- **Impact:**
- **ApplicationController (or a `core.middleware`):** Might include middleware for common error handling or logging request details.
- **All apps:** Developers should implement appropriate error handling for specific operations.
- **`settings.py`:** Contains logging configuration.
- **Deployment:** Requires configuration to ship logs to external services.

8.8. Analytics Integration

- **Description:** Collecting data about user interactions and system events to understand platform usage, identify areas for improvement, and provide insights for DfE.
- **Approach/Technologies:**
- **DfE Analytics Standards:** Adherence to DfE’s standards and potentially using DfE-provided libraries or patterns for data collection and event tracking (e.g., `dfe-analytics` Ruby gem was used previously, a Python equivalent or direct BigQuery integration will be used).

- **Custom Event Tracking:** Specific events (e.g., vacancy searches, job applications started/submitted, job alerts created, PII access) will be tracked with relevant metadata.
- **Data Warehouse:** Data will be sent to a data warehouse, likely Google BigQuery, for storage and analysis, aligning with wider DfE data practices.
- **Anonymization/Pseudonymization:** PII will be handled carefully, with anonymization or pseudonymization applied as appropriate before data is sent to analytics platforms, especially for aggregated reporting.
- **Impact:**
- **Various views and services:** Will need to include code to trigger analytics events at appropriate points.
- **core or analytics app:** Might contain helper functions or services for sending analytics data.
- **Celery tasks:** Could be used for batch sending of analytics events.

8.9. Configuration Management

- **Description:** Managing application settings and secrets across different environments (development, staging, production).
- **Approach/Technologies:**
- **Django settings.py:** The primary location for application configuration.
- **Environment Variables:** Sensitive information (database passwords, API keys, SECRET_KEY, CRYPTOGRAPHY_KEY) and environment-specific settings (e.g., DEBUG mode, ALLOWED_HOSTS) MUST be managed via environment variables. `os.getenv()` will be used to read these.
- **.env files:** For local development, `.env` files (e.g., using `python-dotenv`) will be used to load environment variables, but these files MUST NOT be committed to version control.
- **Cloud Platform Configuration:** In deployed environments, environment variables will be set through the cloud platform's configuration mechanisms (e.g., AWS Parameter Store, Azure App Configuration, Kubernetes ConfigMaps/Secrets).
- **Impact:**
- **teaching_vacancies_py/settings.py:** Central point for defining settings and reading environment variables.
- **Deployment scripts/processes:** Need to ensure environment variables are correctly set in each environment.

- **All developers:** Need to be aware of how to manage local configuration using `.env` files.

8.10. Forms Handling and Validation

- **Description:** Managing user input through web forms, including data validation, cleaning, and error feedback.
- **Approach/Technologies:**
- **Django Forms:** Django's `forms` module (`forms.Form`, `forms.ModelForm`) will be used extensively for creating forms, handling data submission, validating input against defined rules (both built-in and custom validators), and cleaning data.
- **GOV.UK Design System Styling:** Form rendering will strictly adhere to the GOV.UK Design System. This might involve using a Django widget library compatible with GOV.UK styles (e.g., `django-govuk-forms`, `django-crispy-forms` with a GOV.UK template pack) or custom form rendering logic/templates to produce the required HTML structure and CSS classes.
- **Client-side Validation (Progressive Enhancement):** Basic client-side validation (e.g., HTML5 attributes) can be used for immediate feedback, but all critical validation **MUST** be performed server-side.
- **Impact:**
- **All apps with user input:** Will define Django forms for their models or specific operations.
- **Templates:** Will render forms using Django template tags and apply GOV.UK Design System styling.
- **Views:** Will process form submissions, validate data, and handle success/error cases.

8.11. Static Content Management

- **Description:** Managing and serving static informational pages such as help guides, terms and conditions, privacy policy, accessibility statement, etc.
- **Approach/Technologies:**
- **Django Templates:** For simpler static pages, standard Django templates can be used, potentially with a generic view like `TemplateView`.
- **Markdown Rendering:** For content that might be updated by non-developers or requires rich text formatting (e.g., help guides, blog-like

updates), content may be written in Markdown. A Python Markdown library (e.g., `Markdown`, `mistune`) will be used to convert Markdown to HTML for display within Django templates. This was the approach in the Rails app.

- **Static Site Generator (Out of Scope for Core App):** While a full static site generator is out of scope for the main Django application, content might be managed in a way that it **could** be processed by one if needed for a separate content-focused site (e.g., a blog).
- **Impact:**
- **core or pages app:** Might contain views and templates for serving static content.
- **Templates directory:** Will store HTML templates and potentially Markdown files.
- **Deployment:** Static assets (CSS, JS, images for these pages) will be collected and served.

8.12. Data Migration Strategy

- **Description:** The plan and process for migrating data from the existing Ruby on Rails application's database to the new Python/Django application's PostgreSQL database, considering schema changes and data transformations.
- **Approach/Technologies:**
- **ETL Scripts:** Custom Python scripts (potentially using Django's ORM for writing to the new database, and direct DB connection libraries like `psycopg2` for reading from the old) will be developed for Extract, Transform, Load (ETL) operations.
- **Phased Migration (if possible):** Depending on downtime constraints and data dependencies, a phased approach might be considered (e.g., migrate users first, then organisations, then vacancies). However, given the interconnectedness, a single, well-orchestrated cutover period is more likely.
- **Data Mapping:** Detailed mapping from old Rails schema fields to new Django schema fields is required. This was partially covered in the schema design phase.
- **Transformation Logic:** Scripts will handle necessary data transformations (e.g., enum value changes, date formats, rich text conversion if formats differ, handling of encrypted data).

- **Encrypted Data:** For fields encrypted in Rails (e.g., with `attr_encrypted`), a strategy is needed:
 1. Decrypt data from the old database using Rails' encryption keys.
 2. Migrate the plaintext data.
 3. Re-encrypt the data in the new Django system using its new encryption keys and library (e.g., `django-cryptography`). Careful key management is paramount.
- **User Accounts:** User passwords cannot be decrypted. Jobseekers will likely need to reset their passwords upon first login to the new system or be re-authenticated via GOV.UK One Login. Publisher accounts will rely on DSI.
- **Validation and Testing:** Post-migration, extensive data validation checks and testing will be performed to ensure integrity and completeness.
- **Impact:**
- **Significant development effort:** Requires dedicated scripts and thorough testing.
- **Downtime:** A period of downtime for the service will likely be required during the final data migration cutover.
- **All models:** The migration strategy affects all data entities being brought over from the old system.

8.13. Testing Strategy

- **Description:** The overall approach to ensuring the quality, correctness, and reliability of the application through various types of automated and manual testing.
- **Approach/Technologies:**
- **Test Framework:** `pytest` with `pytest-django` for Django integration.
- **Unit Tests:** Focus on testing individual components (functions, methods, classes, Django forms, model methods) in isolation. Mocking will be used to isolate dependencies.
- **Integration Tests:** Verify the interaction between different components (e.g., view processing a form and interacting with a service, service interacting with the database).
- **End-to-End (E2E) Tests:** Test complete user journeys through the web interface, simulating real user behavior. Tools like Selenium, Playwright, or Cypress might be used (choice to be finalized, Playwright is often favored in Python contexts).

- **Code Coverage:** Aim for high code coverage (e.g., >90% for unit tests, >80% overall) using tools like `coverage.py`.
- **Continuous Integration (CI):** Automated tests will be run on every commit/pull request in a CI environment (e.g., GitHub Actions).
- **Static Analysis:** Linters (Flake8/Ruff) and type checkers (MyPy) are part of the quality assurance process.
- **Accessibility Testing:** Automated tools (e.g., Axe) and manual checks against WCAG 2.1 AA guidelines.
- **Security Testing:** Regular vulnerability scans and penetration testing.
- **Impact:**
- **All apps:** Will contain a `tests` directory with test files corresponding to their modules.
- **CI/CD pipeline:** Will integrate test execution and reporting.
- **Development workflow:** Developers will be expected to write tests for new features and bug fixes.